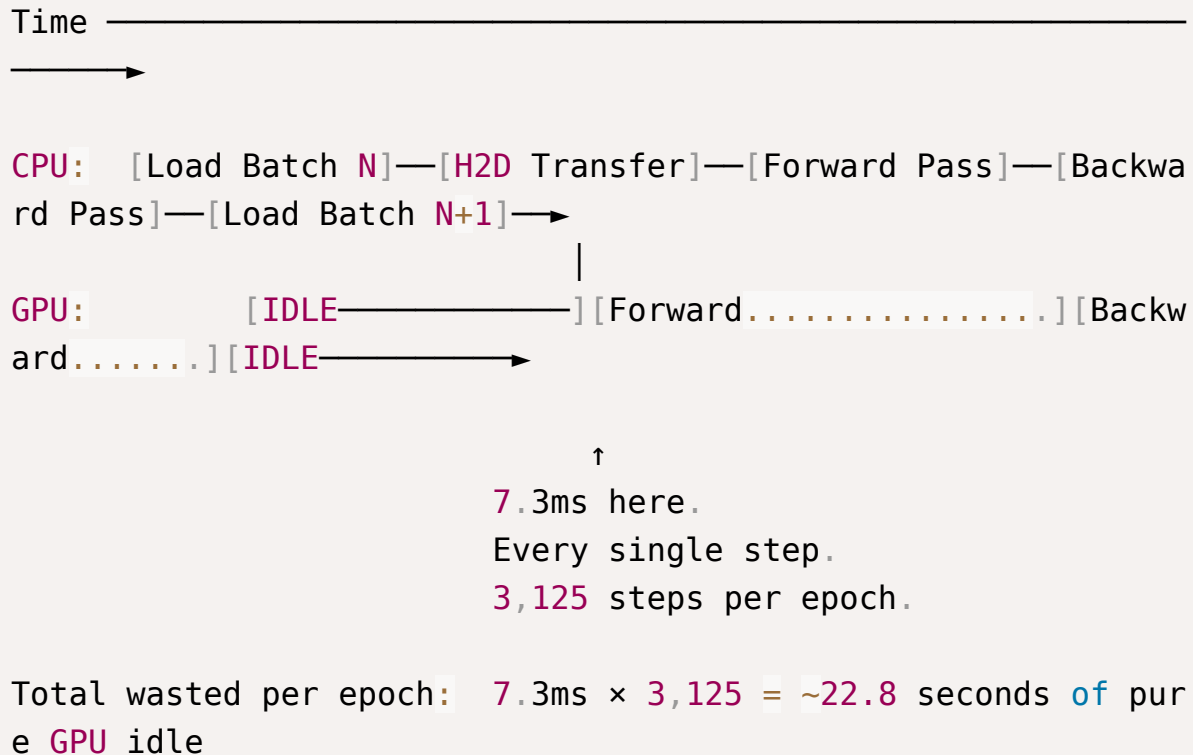


cuda streams

the overall problem was :



AMP cut this to around half by halving the data size, torch.compile didn't touch on this at all as it was a compute optimization rather than a memory one. **CUDA Streams** is the only method that directly eliminates this window.

The core idea, overlap, don't wait

the fundamental insight is that the GPU has two completely independent hardware engines that can run simultaneously

RTX 5070 Ti – Hardware Engines:

Engine 1: CUDA Compute Engine (SMs)

└─ Runs kernels: Conv2d, BN, ReLU, matmuls

Engine 2: DMA Copy Engine

└─ Runs memory transfers: H2D, D2H, D2D

CRITICAL: These are physically separate silicon.

They **do** not share resources.

They can run **AT THE SAME TIME**.

Baseline eager execution never uses both simultaneously.

CUDA Streams is the mechanism that lets you **do** exactly that.

what a cuda stream actually is

A CUDA Stream is a sequential queue of GPU operation that execute in order within the stream, but independently and asynchronously relative to other streams.

ONE STREAM (baseline **default**):

Stream 0: [H2D Batch N]—[Forward N]—[Backward N]—[H2D Batch N+1]→

everything serialized, one thing at a time

TWO STREAMS (**async** prefetching):

Stream 0: [H2D Batch N]—[Forward N]—[Backward N]—[Update]—→

Stream 1: [H2D Batch N+1]—
—[H2D Batch N+2]→

↑

starts **HERE**, **while** Stream 0 is computing

GPU sees both queues simultaneously.
DMA engine handles Stream 1.
Compute engine handles Stream 0.
They run in parallel.

the prefetching pipeline

in practice CUDA Streams async prefetching is implemented with a **double buffer** pattern:

```
# Conceptual implementation for your AI4Mars training loop

stream_compute = torch.cuda.Stream() # handles forward/backward
stream_prefetch = torch.cuda.Stream() # handles H2D transfers

# Pre-load first batch
with torch.cuda.stream(stream_prefetch):
    batch_next = next(dataloader).to('cuda', non_blocking=True)
    #                                     ↑
    #                                     non_blocking=True is the key flag
    #                                     CPU does NOT wait for transfer
    #                                     returns immediately

for batch_current in dataloader:

    # Compute on current batch (Stream 0)
    with torch.cuda.stream(stream_compute):
        output = model(batch_current)
        loss = criterion(output, labels)
        loss.backward()
```

```

optimizer.step()

# SIMULTANEOUSLY prefetch next batch (Stream 1)
with torch.cuda.stream(stream_prefetch):
    batch_next = next(dataloader).to('cuda', non_blocking=True)

# Synchronize – ensure prefetch done before next compute step
stream_compute.wait_stream(stream_prefetch)

```

The idle window isn't reduced - it is completely hidden behind useful compute work.

parallel time complexity change

baseline:

```

T_baseline_step = T_H2D + T_forward + T_backward + T_update
                = 7.3ms + ~12ms + ~24ms + ~2ms
                = ~45.3ms per step

```

under CUDA Streams with perfect overlap:

```

T_streams_step = max(T_H2D, T_forward + T_backward + T_update)
                = max(7.3ms, ~38ms)
                = ~38ms per step

```

Because $T_{\text{compute}} \gg T_{\text{H2D}}$, the transfer is completely hidden.

Savings per step: ~7.3ms

Savings per epoch: 7.3ms × 3,125 steps = ~22.8 seconds

Savings over 10 epochs: ~228 seconds = ~3.8 minutes

This is a fundamentally different complexity class from amp and fusion. AMP speeds up compute. FUsion recuded memory traffic. Streams changes teh execution model form sequential to pipelined.

contention and synchronization overheads under cuda streams

CUDA Streams introduces its own synchronization cost that requires careful analysis for the paper. The critical moment is the stream synchronization barrier between the prefetch and compute stream:

```
stream_compute.wait_stream(stream_prefetch)
```

↑

This is not free. This is a dependency declaration.

What it means:

"Do not begin the next forward pass on stream_compute until stream_prefetch confirms H2D transfer is complete."

If H2D finishes BEFORE backward pass ends → zero stall ✓

If H2D finishes AFTER backward pass ends → stall ✗

this creates a **race condition** between teh two streams that depends on teh relative timing of compute and transfer

Second contention, CPU side DataLoader contention:

Contention Point: CPU DataLoader vs Prefetch Stream

The CPU must:

- | | |
|-----------------------------------|----------------------------------|
| ① Decode next batch from disk/RAM | ← DataLoader workers |
| ② Pin memory (page-lock it) | ← required for non_blocking=True |
| ③ Issue DMA transfer to GPU | ← triggers stream_prefetch |

If DataLoader workers are **slow** (disk **I/O**, augmentation), the prefetch stream stalls waiting **for** pinned **CPU** memory – the **GPU** idle window reappears, just from a different cause.

For AI4Mars **with** large **1024×1024** images and augmentation, **this CPU**-side bottleneck is a real risk.